FPT UNIVERSITY

CAPSTONE PROJECT REPORT

Report 6 – Software User Guides

– Can Tho, November 2026 –

Table of Contents

Table of Tables **2**

Table of Figures **3**

I. Record of Changes **5**

II. Release Package & User Guides **6**

1 Deliverable Package **6**

2 Installation Guides **8**

2.1 System Requirements 8

2.1.1 Client Requirements 8

2.1.2 Server Requirements 8

2.1.3 Software Requirements 9

2.1.4 Network and Configuration Requirements 9

2.2 Installation Instruction 11

3 User Manual **17**

3.1 Overview 17

3.2 Workflow 1: Authentication & Session Management 18

3.3 Workflow 2: Freelancer Identity Verification (eKYC) 20

3.4 Workflow 3: Expert Profile & Portfolio Setup 22

3.5 Workflow 4: Job Posting & Search 23

3.6 Workflow 5: Proposal Submission & Candidate Hiring 25

3.7 Workflow 6: Contract Creation & Escrow Payment 27

3.8 Workflow 7: Milestone Deliverable Submission & Fund Release 28

3.9 Workflow 8: Contract Dispute Resolution 29

3.10 Workflow 9: Ratings & Feedback 30

3.11 Workflow 10: Real-time Messaging & Notifications 31

3.12 Workflow 11: Community Forum & Content Moderation 32

3.13 Workflow 12: Admin System & Master Data Management 34

Table of Tables

1 Record of changes 5
2 Release deliverable package items 7
3 Client hardware and software requirements 8
4 Server hardware requirements 9
5 Server software and runtime dependencies 9
6 Default component ports and production exposure 10
7 Core environment variables configuration 11
8 Seeded demo accounts for verification 17

Table of Figures

1 Downloading Git for Windows, configuring PATH in Setup Wizard, and verifying git version . . . 12

2 Installing Node.js v22 LTS via MSI Wizard and installing PNPM globally in PowerShell . . . 13

3 Installing PostgreSQL 16 on port 5432 and creating frevia_db in pgAdmin 4 . . . 13

4 Installing Docker Desktop and Visual Studio Code and verifying CLI versions . . . 14

5 Cloning the Frevia repository and running pnpm install in VS Code Integrated Terminal . . . 15

6 Configuring .env files, applying Prisma migrations, and seeding RBAC permissions and demo data . . . 16

7 Launching backend and frontend services and verifying the Frevia homepage in browser . . . 17

8 Overall end-to-end feature workflow of the Frevia platform . . . 18

9 Authentication and session management workflow diagram . . . 18

10 User registration, OTP verification, and login interface . . . 19

11 Active device session management and role switching screen . . . 19

12 Freelancer identity verification (eKYC) workflow diagram . . . 20

13 Identity verification document upload interface . . . 21

14 Admin identity verification review queue and inspection modal . . . 21

15 Freelancer profile, AI CV analysis, and portfolio setup workflow diagram . . . 22

16 Freelancer profile editor, skill proficiency selector, and AI CV upload . . . 22

17 Portfolio project editor and public expert showcase . . . 23

18 Job posting and marketplace search workflow diagram . . . 23

19 Client job creation wizard with category, budget, and screening questions . . . 24

20 Freelancer job marketplace search, filters, and bookmarks . . . 24

21 Proposal submission and candidate hiring workflow diagram . . . 25

22 Freelancer proposal submission form with milestone breakdown . . . 26

23 Client proposal comparison and candidate hiring screen . . . 26

24 Contract creation and Stripe escrow payment workflow diagram . . . 27

25 Contract milestone list and Stripe escrow funding checkout . . . 27

26 Milestone deliverable submission and escrow fund release workflow diagram . . . 28

27 Freelancer deliverable upload and Client milestone approval screen . . . 29

28 Contract dispute filing and administrative resolution workflow diagram . . . 29

29 Contract dispute submission form and Admin dispute resolution console . . . 30

30 Two-way contract rating and feedback workflow diagram 30
31 Two-way star rating and contract feedback submission modal 31
32 Real-time WebSocket messaging and notification workflow diagram 31
33 Real-time WebSocket conversation workspace and notification panel 32
34 Community forum posting and content moderation workflow diagram 32
35 Community forum category view, discussion thread, and post editor 33
36 Admin/Moderator content moderation queue and forum trash management . . . 33
37 Administrative system and master data governance workflow diagram 34
38 Admin user management directory and RBAC role-permission matrix 35
39 Admin management consoles for skills, job categories, and advertisement banners 35

I. Record of Changes

Table 1. Record of changes

Date	A* M, D	In charge	Change Description
15/09/2026	A	Pham Kha Vy	Created initial structure for Report 6: Deliverable Package and System Requirements.
05/10/2026	A	Nguyen Tien Thuan	Added Installation Instructions for Docker Compose, PNPM monorepo, Prisma migrations, and Infisical secrets.
19/10/2026	M	Tran Phat Tai	Documented User Manual workflows for Authentication, Sessions, Job Posting, and Proposal Submission.
26/10/2026	M	Nguyen Lu Nhat Huy	Documented workflows for Real-time Messaging, Notifications, Community Forum, and Content Moderation.
01/11/2026	M	Vo Nhut Tin	Documented workflows for Contract Creation, Stripe Escrow Payment, Milestone Deliverables, and Dispute Resolution.
04/11/2026	M	Pham Kha Vy, Nguyen Tien Thuan	Added workflows for eKYC Identity Verification, Expert Profile, and Admin Master Data Management; finalized Release v1.0.

**A - Added M - Modified D - Deleted*

II. Release Package & User Guides

1. Deliverable Package

This section lists all project documents, source code repositories, database migration and seed scripts, test suites, and presentation materials included in the final release package of the **Frevia – Verified Freelance Marketplace Platform** (Project Code: `SEP490_04`).

Table 2. Release deliverable package items

No.	Deliverable Item	Description
1	Project Schedule & Tracking (`v2.0`)	Master work breakdown structure (WBS), 16-week sprint schedule across 5 sprints, resource allocation for 5 team members (372 man-days), and milestone tracking documented in `docs/Report 2/`.
2	Project Backlog (`v2.0`)	Complete product and sprint backlog covering 14 functional modules, 84 use-case features, and 214 SMAR items documented in `docs/Report 2/` and `docs/Report 3/`.
3	Source Codes (`v1.0.0`)	Full Turborepo/PNPM monorepo source code including `apps/api` (NestJS 11 REST API & Socket.IO Gateway), `apps/web` (Next.js 16 App Router Frontend), and `packages/types` (Shared Zod validation schemas & TypeScript contracts).
4	Database Script(s) (`v1.0.0`)	Prisma relational schema (`apps/api/prisma/schema.prisma`), 31 SQL migration scripts (`apps/api/prisma/migrations/`), and deterministic TypeScript seed scripts (`apps/api/src/seed-datas/`).
5	Final Report Documents (`v2.0`)	Complete Capstone Project documentation suite: Report 1 (Project Introduction), Report 2 (Project Management Plan), Report 3 (Software Requirement Specification), Report 4 (Software Design Document), Report 5 (Software Test Documentation), and Report 6 (Software User Guides).
6	Test Cases Document (`v2.0`)	Unit test workbook `Report5_Unit Test.xls` (84 service functions, 507 unit test cases) and system test workbook `Report5_Test Report.xlsx` (14 modules, 411 system test cases) in `docs/Report 5/`.
7	Defects List (`v1.0`)	Sprint defect tracking log recording 17 identified defects (2 Critical, 6 Major, 9 Minor) with root cause analysis and 100% verification closure in `docs/Report 5/`.
8	Issues List (`v1.0`)	Project risk and technical issue register covering third-party webhook idempotency, RBAC session synchronization, and cloud storage configuration.
9	Deployment Configuration (`v1.0.0`)	Production and staging container orchestration files (`docker-compose.yml`, `apps/api/Dockerfile`, `apps/web/Dockerfile`) and environment templates.
10	Capstone Presentation Slide (`v1.0`)	Final Capstone Defense slide deck summarizing business value, system architecture, test results, and live demonstration scenarios.

2. Installation Guides

2.1 System Requirements

2.1.1 Client Requirements

End users (Clients, Freelancers, Experts, Moderators, and Administrators) access the Frevia platform through a standard web browser without installing proprietary client software.

Table 3. Client hardware and software requirements

Category	Minimum Requirement	Recommended Specification
Processor (CPU)	Dual-core 1.8 GHz (x64 or ARM64)	Quad-core 2.4 GHz or higher
Memory (RAM)	4 GB RAM	8 GB RAM or higher
Display Resolution	1280×720 pixels (HD)	1920×1080 pixels (Full HD)
Operating System	Windows 10, macOS 12, Ubuntu 20.04, iOS 16, or Android 12	Windows 11, macOS 14+, Ubuntu 22.04+
Web Browser	Google Chrome 120+, Mozilla Firefox 120+, Microsoft Edge 120+, Safari 16+	Latest stable Google Chrome or Microsoft Edge with JavaScript and Cookies enabled
Network Connection	Broadband 10 Mbps	Broadband 25 Mbps+ (low latency for real-time WebSocket chat)

2.1.2 Server Requirements

The backend API, Next.js frontend server, PostgreSQL database, and optional Ollama AI container can be deployed on a single host using Docker Compose or distributed across cloud instances.

Table 4. Server hardware requirements

Component	Minimum (Local / Staging)	Recommended (Production)
Processor (CPU)	4 vCPUs (x86_64 / ARM64)	8 vCPUs (Dedicated cloud instance)
Memory (RAM)	8 GB RAM	16 GB RAM (especially when running Ollama AI locally)
Disk Storage	30 GB SSD free space	100 GB NVMe SSD with automated daily snapshots
Operating System	Ubuntu Server 22.04 LTS / Windows 11 with WSL2	Ubuntu Server 24.04 LTS (64-bit)

2.1.3 Software Requirements

Table 5 lists the required runtime engines, frameworks, and CLI tools to build and run the platform.

Table 5. Server software and runtime dependencies

Software / Tool	Version	Purpose
Node.js	`>= 20.x LTS`	JavaScript/TypeScript runtime for NestJS and Next.js
PNPM	`9.0.0+`	Workspace package manager for the monorepo
Turborepo	`2.10.0`	Monorepo build system and task orchestrator
PostgreSQL	`16.x`	Primary relational database engine
Prisma ORM	`7.8.0`	Type-safe database client and migration engine
Docker & Docker Compose	`24.0+ / v2.24+`	Containerization for frontend, backend, and Ollama services
Infisical CLI (Optional)	`0.28+`	Centralized environment secret injection (`infisical run`)
Stripe CLI (Optional)	`1.20+`	Forwarding local Stripe webhook events during testing

2.1.4 Network and Configuration Requirements

Table 6 specifies the default network ports used by each component in both containerized (`docker-compose.yml`) and local development modes.

Table 6. Default component ports and production exposure

Component	Port (Docker / Dev)	Production Exposure	Description
Next.js Web Portal (`frontend`)	`3003` / `3001`	Public (HTTPS 443 via Reverse Proxy)	Serves Client, Freelancer, Forum, and Admin web interfaces. Maps host port `3003` to container port `3000`.
NestJS Backend API (`backend`)	`3002` / `3000`	Public via API Gateway / Proxy	Serves REST endpoints (`/api/*`) and Socket.IO namespaces (`/chat`, `/notifications`). Maps host port `3002` to container port `3000`.
PostgreSQL Database	`5432`	Internal Only (Private VPC)	Primary relational store accessed via Prisma connection pool (`DIRECT_URL`).
Ollama AI Engine (`ollama`)	`11434` / `8000`	Internal Only (Docker Network)	Local LLM inference service used for AI CV skill extraction.

Table 7 summarizes the mandatory environment variables configured in `apps/api/.env` and `apps/web/.env`.

Table 7. Core environment variables configuration

Variable Name	Service	Description
DIRECT_URL	API	PostgreSQL connection string for Prisma Client and migrations.
PORT	API	Internal HTTP listening port for NestJS (default: 3000).
NEXT_URL	API	Allowed frontend origin URL for CORS and OAuth redirects.
ACCESS_TOKEN_SECRET	API	HMAC secret key for signing short-lived JWT access tokens.
REFRESH_TOKEN_SECRET	API	HMAC secret key for signing persisted JWT refresh tokens.
ACCESS_TOKEN_EXPIRES_IN	API	Access token validity window (e.g., 15m or 1h).
REFRESH_TOKEN_EXPIRES_IN	API	Refresh token validity window (e.g., 7d).
OTP_EXPIRES_IN	API	Verification OTP expiration window (e.g., 5m).
EMAIL_USERNAME / EMAIL_PASSWORD	API	SMTP credentials used by Nodemailer to send registration and password recovery OTP emails.
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET	API	Google Cloud OAuth 2.0 credentials for social login.
CLOUDINARY_CLOUD_NAME / API_KEY / API_SECRET	API	Cloudinary credentials for uploading avatars, portfolios, CVs, and eKYC identity images.
STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET	API	Stripe API secret key and webhook signature verification key for milestone escrow payments.
APP_URL / NESTJS_API_URL	Web	Public web URL and backend API base URL consumed by Next.js server and client components.

2.2 Installation Instruction

This section provides a complete, step-by-step installation guide starting from a clean machine with no pre-installed development tools through to launching and verifying the Frevia platform. Every interactive step on the screen is highlighted with a numbered red box corresponding to the instructions below.

Step 1: Download and Install Git

1. Open a web browser, navigate to `https://git-scm.com/downloads/win`, and click **Click here to download (64-bit Git for Windows Setup) (1)**.
2. Run the downloaded `Git-2.47.x-64-bit.exe` installer, select **Git from the command line and also from 3rd-party software (2)**, and click **Next / Install (3)**.
3. Open **Windows PowerShell** and execute `git -version` and `git config -global` to verify the installation **(4)**:

```
git --version
git config --global user.name "Nguyen Tien Thuan"
git config --global user.email "thuanntce181024@fpt.edu.vn"
```

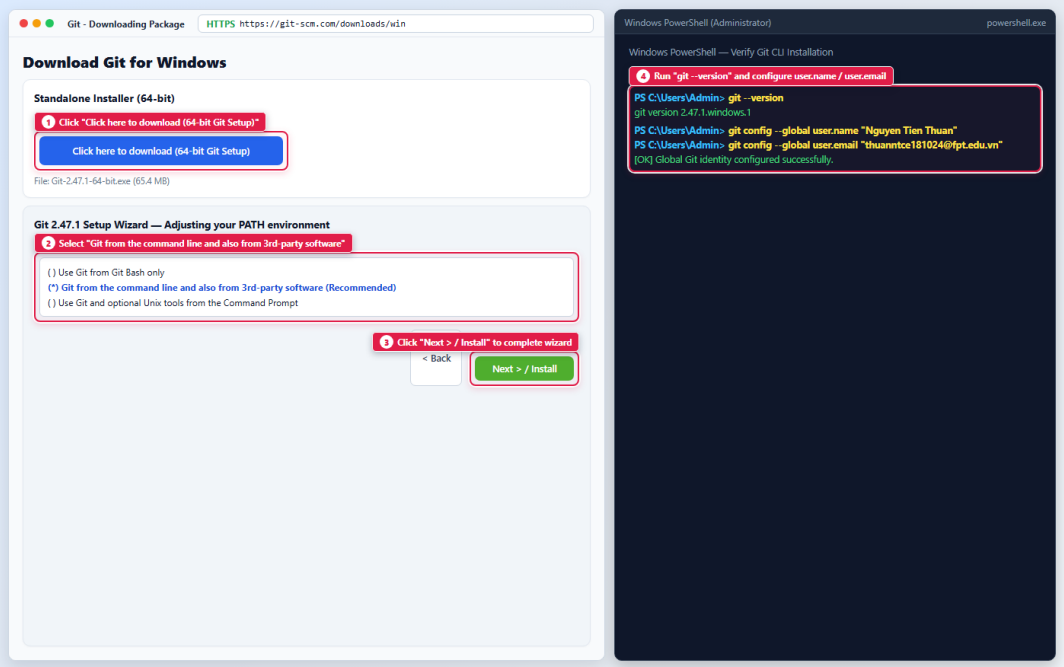


Figure 1. Downloading Git for Windows, configuring PATH in Setup Wizard, and verifying git version

Step 2: Download and Install Node.js LTS and PNPM

1. Navigate to `https://nodejs.org/en/download/prebuilt-installer`, select **Node.js v22.x (LTS)** for Windows x64, and click **Download Node.js LTS (.msi) (1)**.
2. Launch the MSI installer, check the option to install necessary tools, and click **Next / Install (2)**.
3. Open PowerShell, verify `node -v` and `npm -v` **(3)**, then install the **PNPM** workspace package manager globally and verify `pnpm -v` **(4)**:

```
node -v
npm -v
npm install -g pnpm@9.0.0
pnpm -v
```

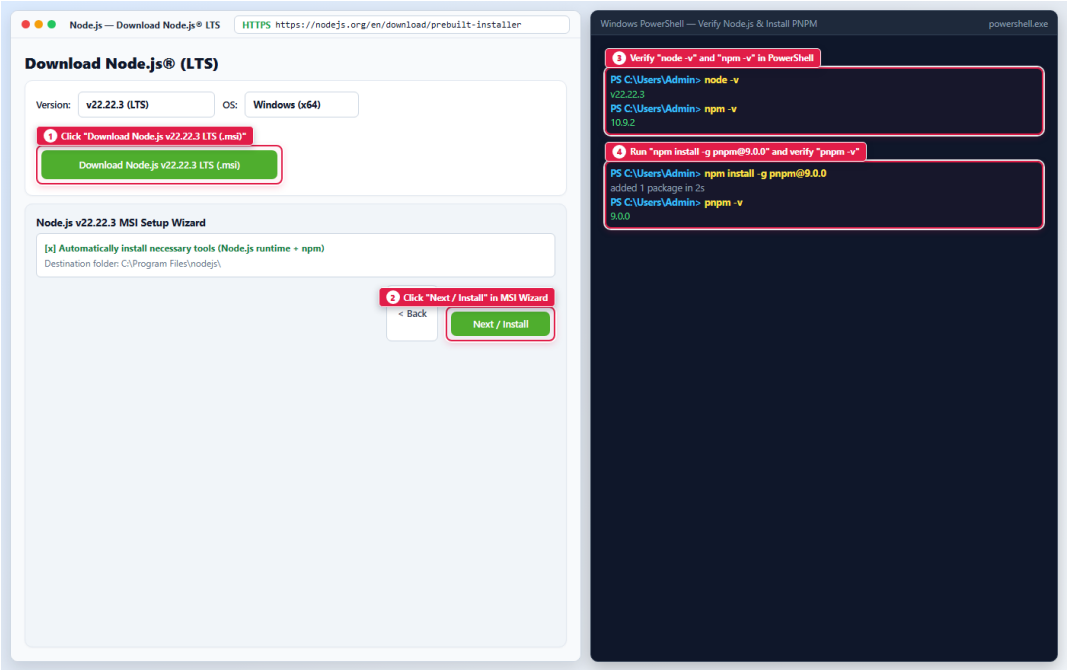


Figure 2. Installing Node.js v22 LTS via MSI Wizard and installing PNPM globally in PowerShell

Step 3: Install PostgreSQL 16 and Create the `frevia_db` Database

1. Download the PostgreSQL 16.x Windows x86-64 installer from `https://www.postgresql.org/downloa` and complete the wizard using port `5432` and superuser `postgres` **(2)**.
2. Launch **pgAdmin 4**, expand **Servers** → **PostgreSQL 16**, right-click **Databases**, and select **Create** → **Database... (1)**.
3. Enter `frevia_db` in the **Database** field **(3)** and click **Save (4)**.

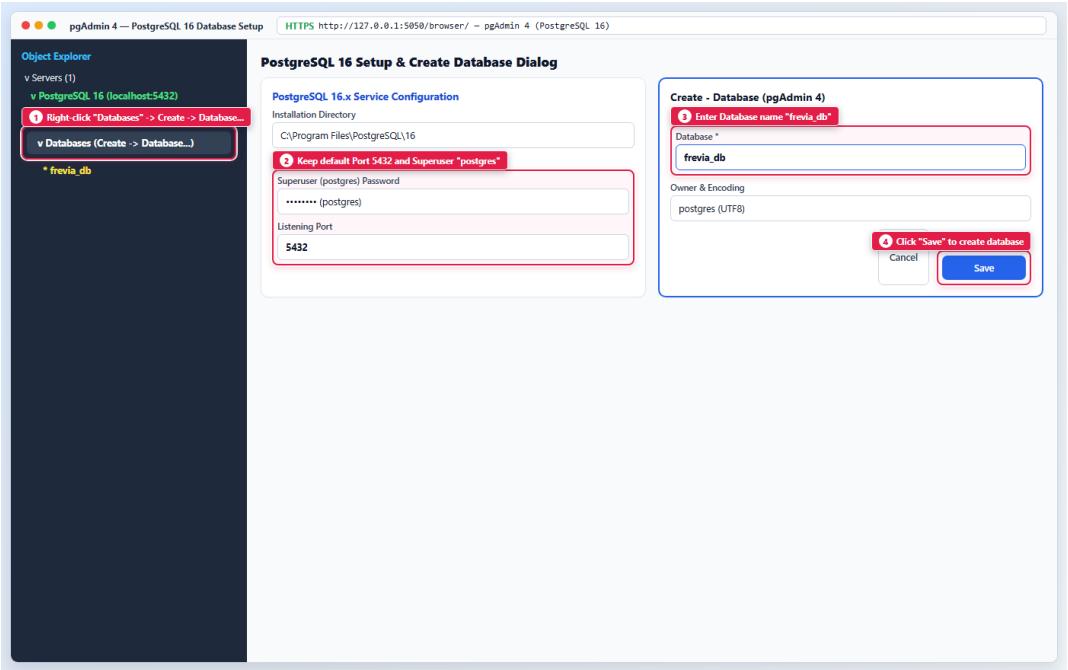


Figure 3. Installing PostgreSQL 16 on port 5432 and creating frevia_db in pgAdmin 4

Step 4: Install Docker Desktop and Visual Studio Code

1. Navigate to `https://www.docker.com/products/docker-desktop/`, click **Download for Windows - AMD64 (1)**, install Docker Desktop with the WSL 2 backend, and confirm the engine status shows **Engine running**.
2. Navigate to `https://code.visualstudio.com/`, click **Download VS Code for Windows x64 (2)**, and complete the installation.
3. Open PowerShell and verify `docker -version`, `docker compose version`, and `code -version` **(3)**:

```
docker --version
docker compose version
code --version
```

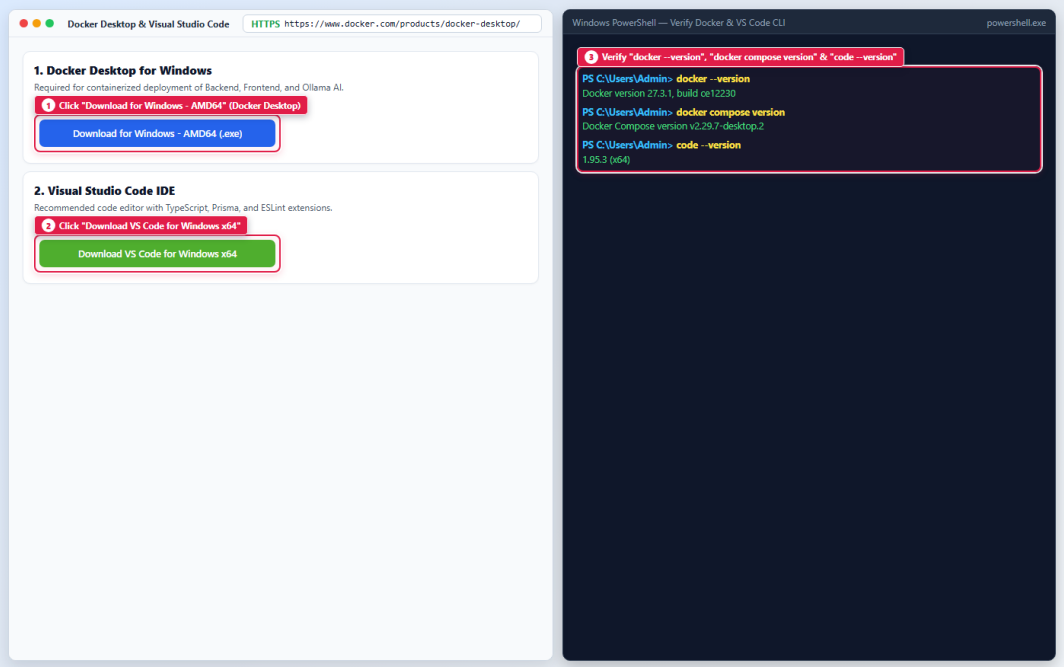


Figure 4. Installing Docker Desktop and Visual Studio Code and verifying CLI versions

Step 5: Clone the Repository and Install Monorepo Dependencies

1. Open Visual Studio Code, open the Integrated Terminal (`Ctrl + ~`), and verify the workspace directory structure (`apps/api`, `apps/web`, `packages/types`, `packages/ui`) **(1)**.
2. Clone the Frevia repository and change into the project root directory **(2)**.
3. Run `pnpm install` to install all workspace dependencies **(3)**:

```
git clone https://github.com/frevia-platform/frevia.git
cd frevia
pnpm install
```

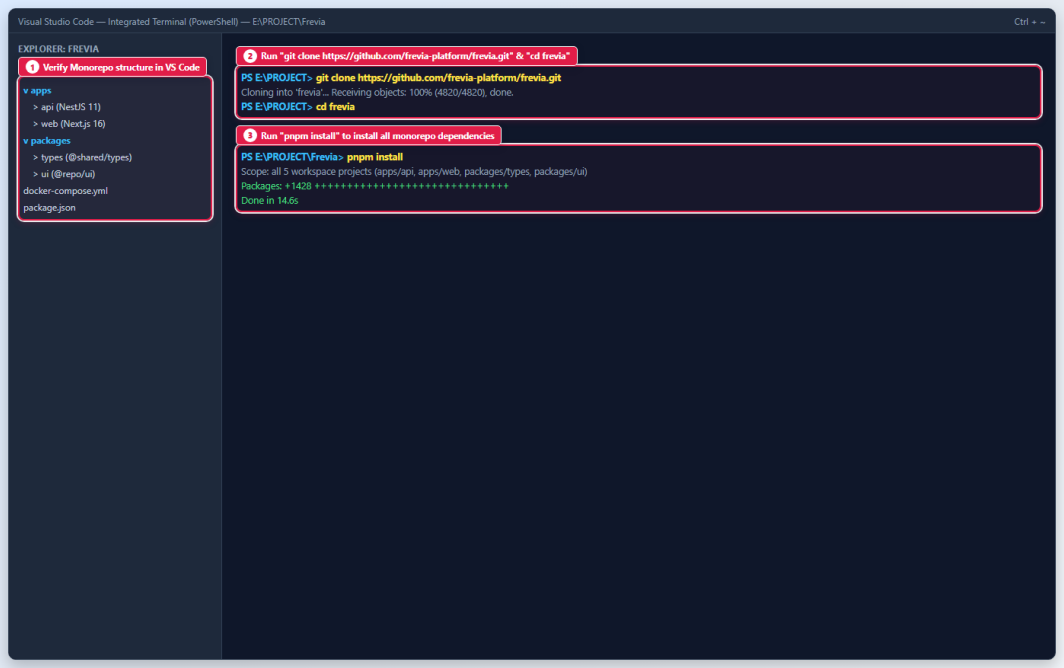


Figure 5. Cloning the Frevia repository and running pnpm install in VS Code Integrated Terminal

Step 6: Configure Environment Variables, Run Prisma Migrations, and Seed Data

1. Create `apps/api/.env` and `apps/web/.env` in VS Code and configure the PostgreSQL connection string (`DIRECT_URL`), JWT secrets, Stripe keys, and Cloudinary credentials **(1)**:

```
# Backend environment (apps/api/.env)
NODE_ENV=development
PORT=3000
NEXT_URL=http://localhost:3001
DIRECT_URL="postgresql://postgres:postgres@localhost:5432/frevia_db?schema=
    public"
ACCESS_TOKEN_SECRET="replace-with-strong-access-secret"
REFRESH_TOKEN_SECRET="replace-with-strong-refresh-secret"
ACCESS_TOKEN_EXPIRES_IN="1h"
REFRESH_TOKEN_EXPIRES_IN="7d"
OTP_EXPIRES_IN="5m"
OTP_ATTEMPT_WINDOW="15m"
EMAIL_USERNAME="your-smtp-email@gmail.com"
EMAIL_PASSWORD="your-google-app-password"
GOOGLE_CLIENT_ID="your-google-client-id"
GOOGLE_CLIENT_SECRET="your-google-client-secret"
GOOGLE_REDIRECT_URL="http://localhost:3000/api/auth/google/callback"
CLOUDINARY_CLOUD_NAME="your-cloud-name"
CLOUDINARY_API_KEY="your-cloudinary-key"
CLOUDINARY_API_SECRET="your-cloudinary-secret"
STRIPE_SECRET_KEY="sk_test_..."
STRIPE_WEBHOOK_SECRET="whsec_..."
```

```
# Frontend environment (apps/web/.env)
```

```
NODE_ENV=development
APP_URL=http://localhost:3001
NESTJS_API_URL=http://localhost:3000
NEXT_PUBLIC_API_URL=http://localhost:3000
```

2. Generate the Prisma client and apply all 31 relational migrations to `frevia_db` **(2)**.
3. Execute the seed scripts to populate default roles, 142 RBAC permissions, job categories, skills, and demo user accounts **(3)**:

```
cd apps/api
npx prisma generate
npx prisma migrate deploy
pnpm seed:permissions
pnpm seed:job-categories
pnpm seed
cd ../..
```

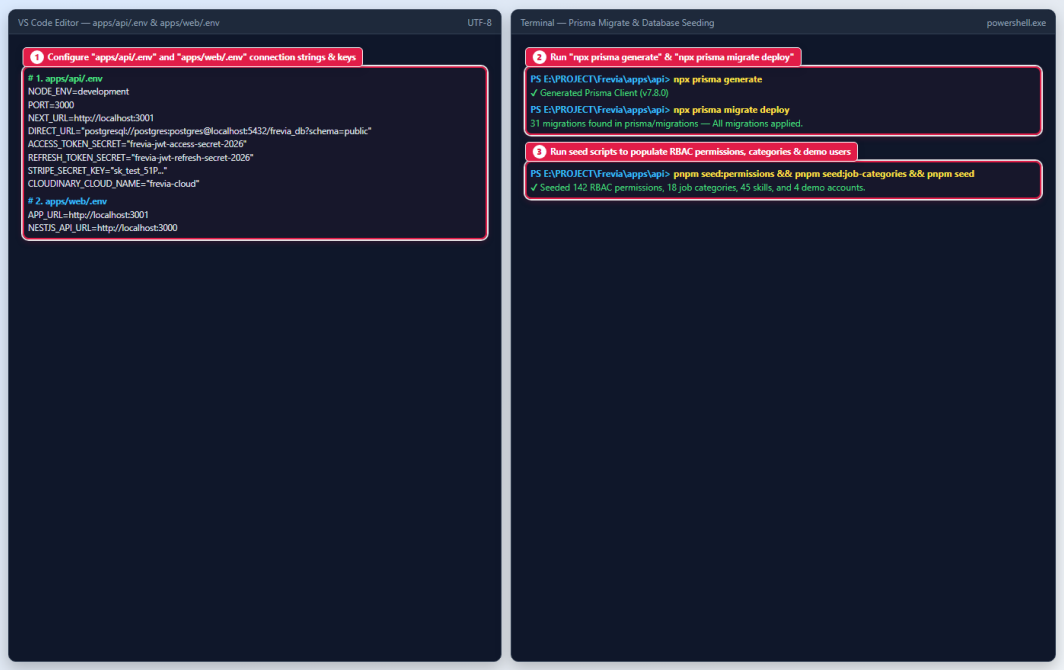


Figure 6. Configuring .env files, applying Prisma migrations, and seeding RBAC permissions and demo data

Step 7: Launch the Application and Verify in Browser

1. **Option A – Docker Compose Mode:** Run `docker compose up -d -build` to launch the backend (:3002), frontend (:3003), and Ollama AI containers **(1)**.
2. **Option B – Local Development Mode:** Run `pnpm -filter api run dev:local` (port 3000) and `pnpm -filter web exec next dev -port 3001` **(2)**.
3. Open a web browser at `http://localhost:3003` (or `http://localhost:3001`) **(3)** and click **Get Started / Register** or **Sign In with Demo Account (4)** using the credentials in Table 8.

```
# Option A: Docker Compose Production / Staging Stack
```

```
docker compose up -d --build
docker compose ps

# Option B: Local Development Mode
pnpm --filter api run dev:local
pnpm --filter web exec next dev --port 3001
```

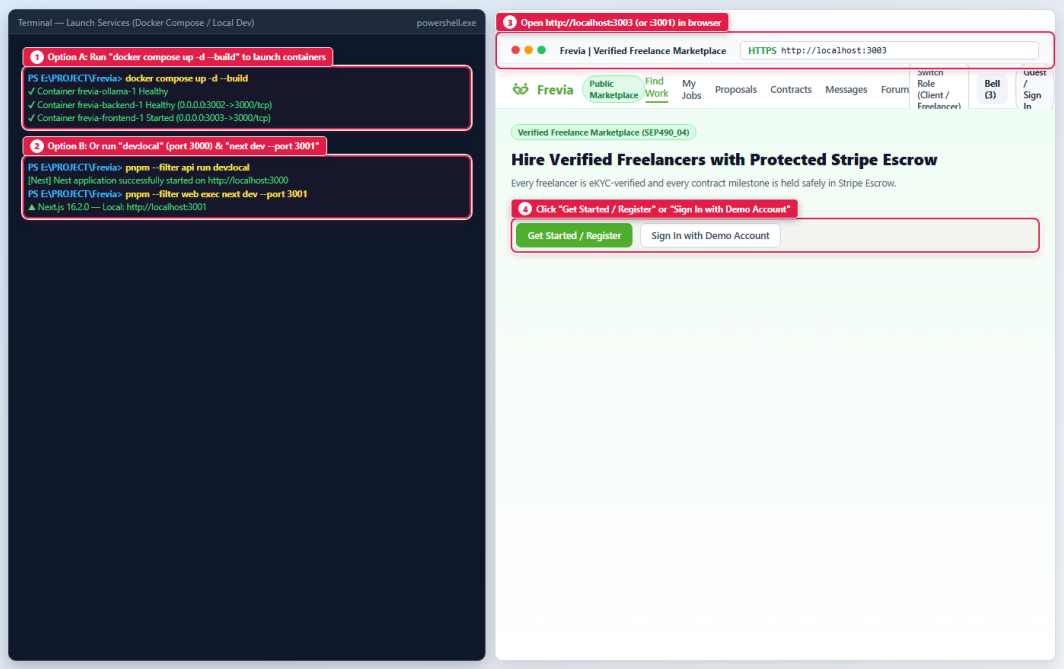


Figure 7. Launching backend and frontend services and verifying the Frevia homepage in browser

Table 8. Seeded demo accounts for verification

Role	Email Address	Password	Default Workspace
Administrator	`admin@frevia.local`	`Admin@123456`	`/admin/users`
Moderator	`moderator@frevia.local`	`Mod@123456`	`/admin/moderation`
Client	`client.demo@frevia.local`	`Client@123456`	`/client/jobs`
Freelancer	`freelancer.demo@frevia.local`	`Freelancer@123456`	`/freelancer/find-work`

3. User Manual

3.1 Overview

Frevia is a verified freelance marketplace platform that connects **Clients** (employers and businesses seeking specialized talent) with verified **Freelancers / Experts** under the governance of **Administrators** and **Moderators**. The platform organizes its features into four dedicated role workspaces:

- **Identity & Profile Layer:** Multi-role JWT authentication, active session control, eKYC Citizen ID/Passport verification, freelancer portfolio management, and AI-assisted CV skill extraction.
- **Marketplace & Bidding Layer:** Job posting with category/skill taxonomy and screening questions, advanced job search, saved searches, and milestone-based proposal bidding.
- **Contract, Escrow & Dispute Layer:** Contract generation from accepted proposals, Stripe escrow milestone funding, deliverable submission and approval, automated fund release, two-way star ratings, and administrative dispute resolution.
- **Community & Governance Layer:** Real-time WebSocket 1-on-1 messaging, instant notifications, community discussion forum with content moderation, and back-office master data management (Users, Roles, Permissions, Banners, Skills, and Job Categories).

Figure 8 illustrates the end-to-end functional workflow across all platform roles.

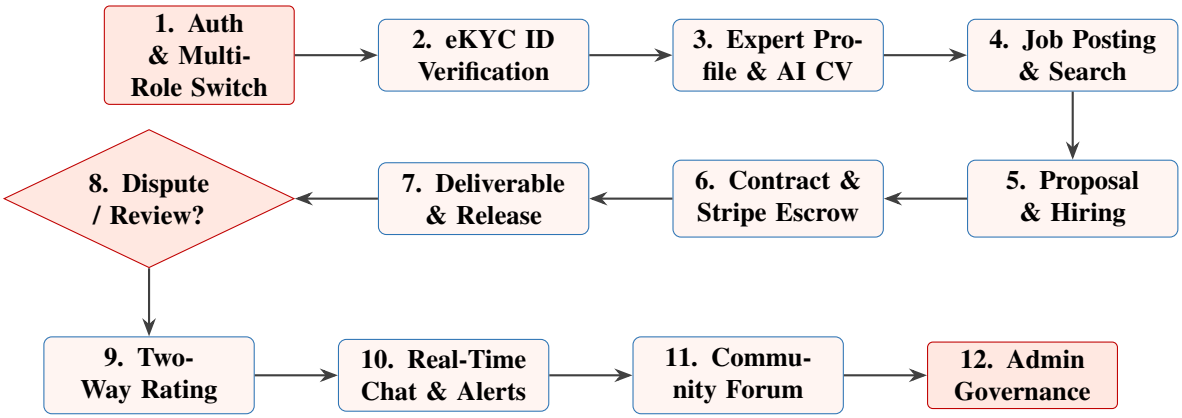


Figure 8. Overall end-to-end feature workflow of the Frevia platform

3.2 Workflow 1: Authentication & Session Management

Purpose and Roles: This workflow guides users through registering an account, verifying their email via a 6-digit OTP code, signing in via email/password or Google OAuth, switching between active roles (`CLIENT` and `FREELANCER`), and revoking active device sessions. **Involved Roles:** Guest, Client, Freelancer, Moderator, Administrator.

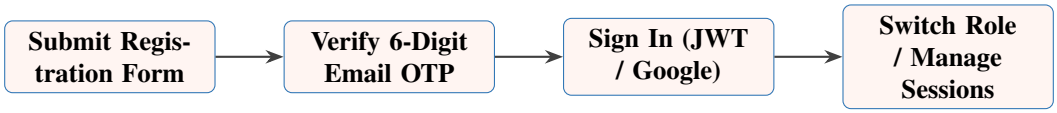


Figure 9. Authentication and session management workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Register Account:** Navigate to `/register`. Select the initial primary role (**Client** or **Freelancer**), enter **Full Name**, **Email Address**, **Password** (minimum 8 characters), and **Confirm Password**, then click **Create Account**.
2. **Step 2 – Verify Email OTP:** Check your inbox for the 6-digit verification code sent by Frevia. Enter the 6 digits into the OTP verification prompt and click **Verify OTP**.
3. **Step 3 – Log In:** Navigate to `/login`. Enter your registered **Email** and **Password** and click **Sign In**, or click **Continue with Google** to authenticate via Google OAuth 2.0.

4. **Step 4 – Join or Switch Role:** Click the profile menu in the top-right navigation bar. Select **Switch to Freelancer** or **Switch to Client** (or click **Join Role** if activating the secondary role for the first time). The workspace navigation updates immediately.
5. **Step 5 – Manage Device Sessions:** Navigate to `/sessions`. Review the table of active sessions displaying browser user-agent, IP address, creation time, and expiry time. Click **Revoke** next to any unrecognized device to invalidate its refresh token immediately.

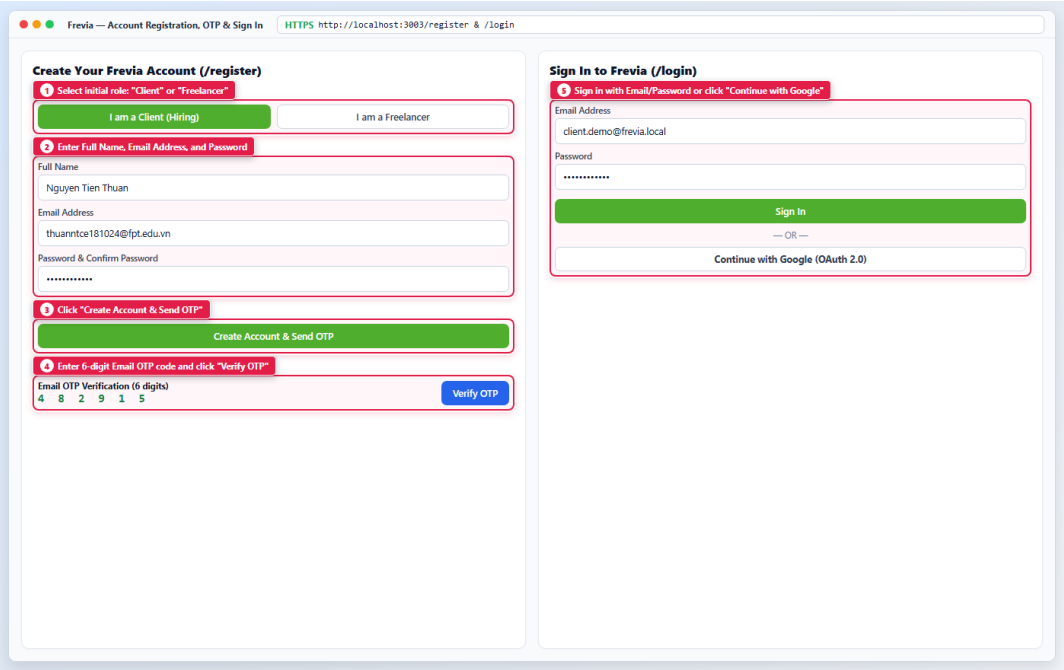


Figure 10. User registration, OTP verification, and login interface

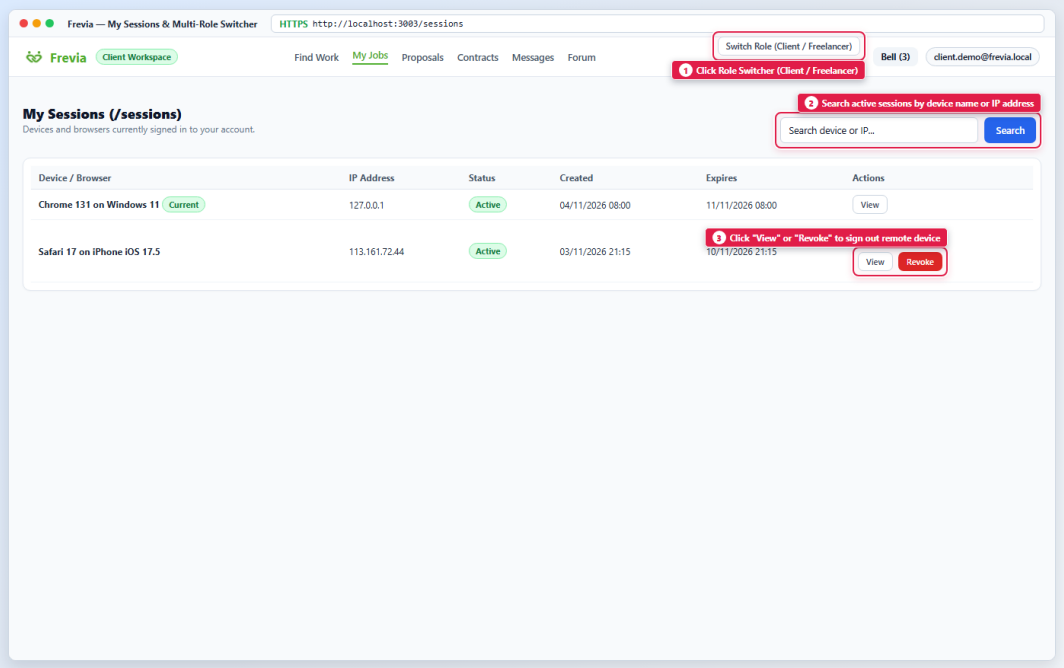


Figure 11. Active device session management and role switching screen

Notes and Common Errors:

- **Expired OTP Code:** Each OTP code expires after 5 minutes. Click **Resend OTP** if the timer expires.
- **OTP Rate Limit:** Exceeding the maximum OTP verification attempts within 15 minutes temporarily locks OTP submission; wait for the cooldown window before retrying.

3.3 Workflow 2: Freelancer Identity Verification (eKYC)

Purpose and Roles: This workflow enables users to verify their legal identity by uploading government-issued identification documents for administrative inspection, earning a verified trust badge on their profile. **Involved Roles:** Freelancer / Client (Requester) and Administrator (Reviewer).

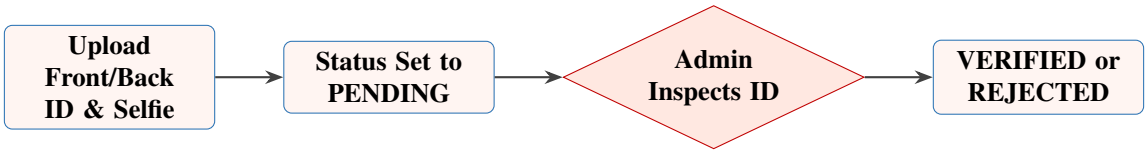


Figure 12. Freelancer identity verification (eKYC) workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Open Identity Verification Panel:** Sign in and navigate to `/account-profile`. Scroll to the **Identity Verification (eKYC)** section.
2. **Step 2 – Upload Identity Images:** Select the document type (**Citizen ID Card** or **Passport**), upload a clear image of the **Front Side**, **Back Side**, and a **Selfie Holding ID** (JPEG/PNG/WebP, maximum 5 MB per file).
3. **Step 3 – Submit Verification Request:** Click **Submit Verification**. The verification status badge changes to `PENDING`.
4. **Step 4 – Admin Inspection:** Sign in as an Administrator and navigate to `/admin/identity-verificatio` Filter the queue by **Pending** status and click **View Details** on a submission.
5. **Step 5 – Approve or Reject:** Inspect the uploaded images. Click **Approve** to grant the `VERIFIED` badge, or click **Reject**, enter a mandatory rejection reason (e.g., blurred ID number), and confirm.

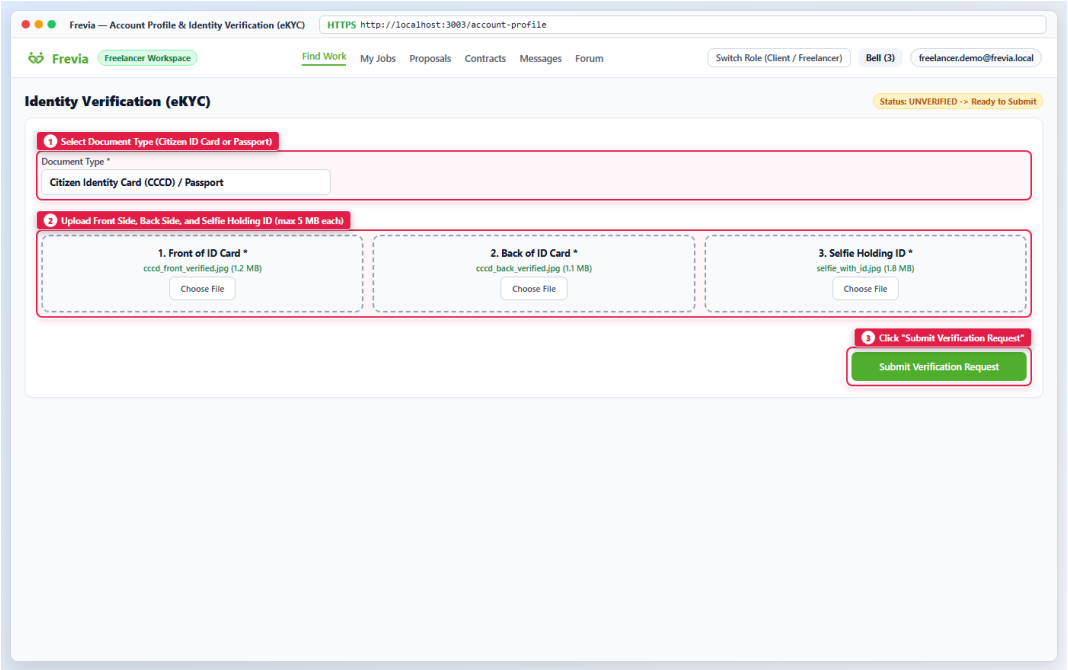


Figure 13. Identity verification document upload interface

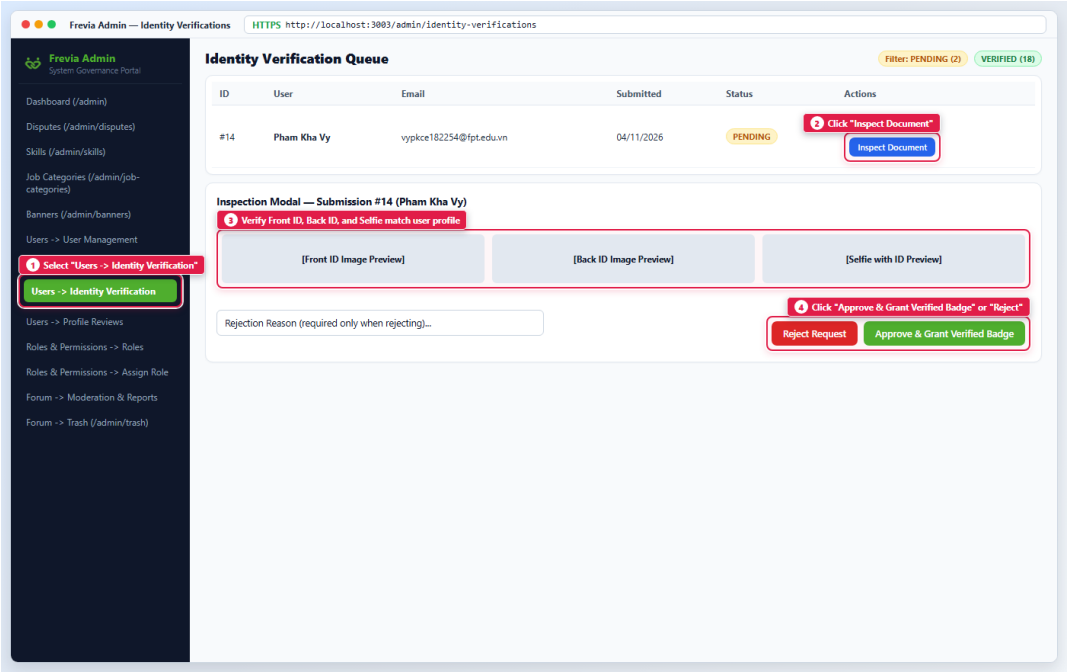


Figure 14. Admin identity verification review queue and inspection modal

Notes and Common Errors:

- **Duplicate Pending Request:** Users cannot submit a second eKYC request while an existing request is in `PENDING` status or already `VERIFIED`.
- **Re-submission After Rejection:** If an Administrator rejects a submission, the user sees the rejection reason on `/account-profile` and can upload corrected images immediately.

3.4 Workflow 3: Expert Profile & Portfolio Setup

Purpose and Roles: This workflow explains how freelancers configure their professional profile, extract skills from a PDF curriculum vitae using AI, add portfolio projects, and submit profile revisions for administrative approval. **Involved Roles:** Freelancer, Expert, Administrator.

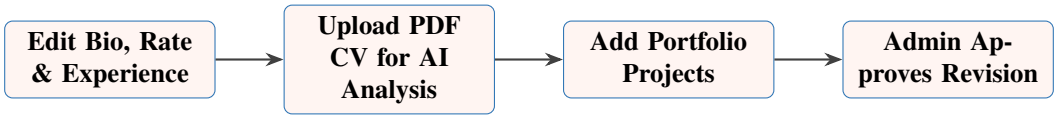


Figure 15. Freelancer profile, AI CV analysis, and portfolio setup workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Open Freelancer Profile Editor:** Switch to the **Freelancer** role and navigate to `/freelancer/profile`.
2. **Step 2 – Update Professional Details:** Enter your **Professional Title**, **Hourly Rate**, **Availability Status**, **Overview Bio**, **Education**, **Certifications**, **Languages**, and **Work Experience**.
3. **Step 3 – Select Skills & Proficiency:** Search the master skill list, attach relevant skills, and set the proficiency level (`1` to `10`) for each skill.
4. **Step 4 – Upload CV for AI Analysis:** In the **Resume / CV** panel, upload a PDF file (up to 10 MB) and click **Analyze CV with AI**. Review the extracted skills and experience summary and click **Apply Suggestions**.
5. **Step 5 – Add Portfolio Item:** In the **Portfolio** tab, click **Add Project**, enter the **Project Title**, **Description**, **Project URL**, and cover thumbnail, then click **Save Project**.
6. **Step 6 – Admin Revision Approval:** If profile revision moderation is required, an Administrator navigates to `/admin/profile-revisions`, inspects the diff, and clicks **Approve Revision**.

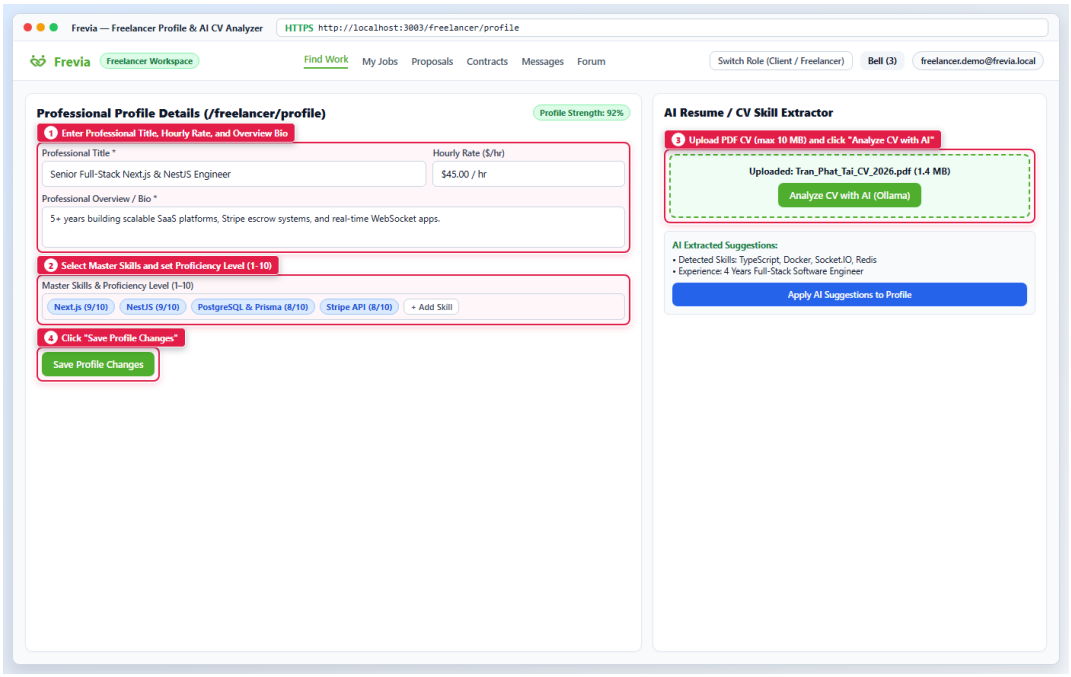


Figure 16. Freelancer profile editor, skill proficiency selector, and AI CV upload

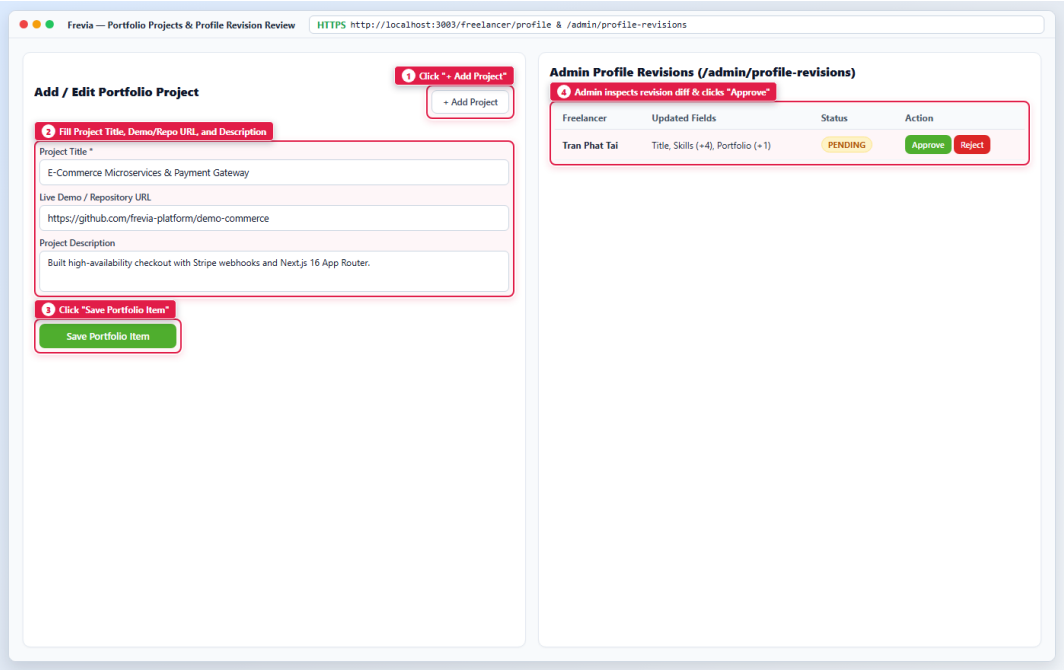


Figure 17. Portfolio project editor and public expert showcase

Notes and Common Errors:

- **CV File Format:** Only valid `application/pdf` files up to 10 MB are accepted for CV upload and AI analysis.
- **Profile Strength Meter:** Completing bio, skills, portfolio items, and eKYC verification raises the profile strength score toward 100%, improving ranking in client talent searches.

3.5 Workflow 4: Job Posting & Search

Purpose and Roles: This workflow covers how Clients create and publish structured job posts and how Freelancers discover, filter, bookmark, and save job searches. **Involved Roles:** Client, Freelancer.

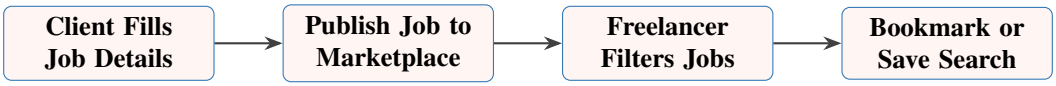


Figure 18. Job posting and marketplace search workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Open Job Creation Form (Client):** In the **Client** workspace, navigate to `/client/jobs/new`.
2. **Step 2 – Configure Job Post:** Enter the **Job Title**, select a **Job Category**, write the detailed **Description**, choose **Experience Level** (`ENTRY`, `INTERMEDIATE`, `EXPERT`), select **Payment Type** (`FIXED` or `HOURLY`), enter the **Budget** amount, attach required **Skills**, and optionally add **Screening Questions**.
3. **Step 3 – Publish Job:** Click **Publish Job** to make the listing live immediately (or **Save as Draft** to edit later at `/client/jobs`).

4. **Step 4 – Search and Filter Jobs (Freelancer):** Switch to the **Freelancer** workspace and navigate to `/freelancer/find-work`. Enter keywords or filter by **Job Category**, **Skills**, **Budget Range**, and **Experience Level**.
5. **Step 5 – Bookmark Job or Save Search:** Click the **Bookmark** icon on any job card to save it to `/freelancer/bookmarks`, or click **Save Search** to store the current filter combination in `/freelancer/saved-searches`.

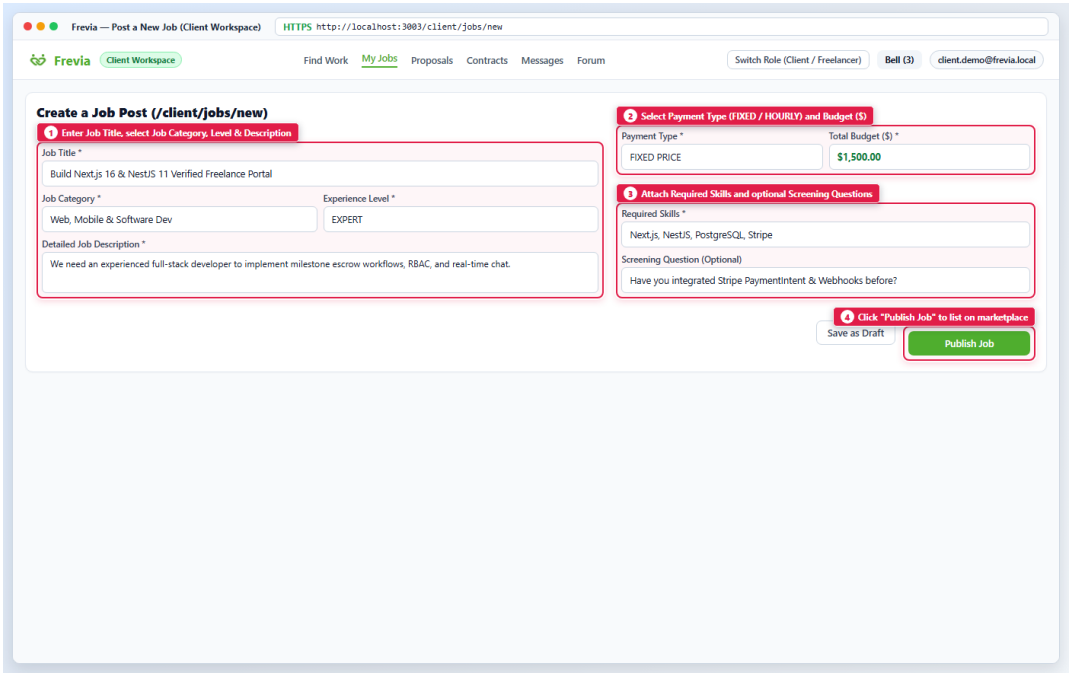


Figure 19. Client job creation wizard with category, budget, and screening questions

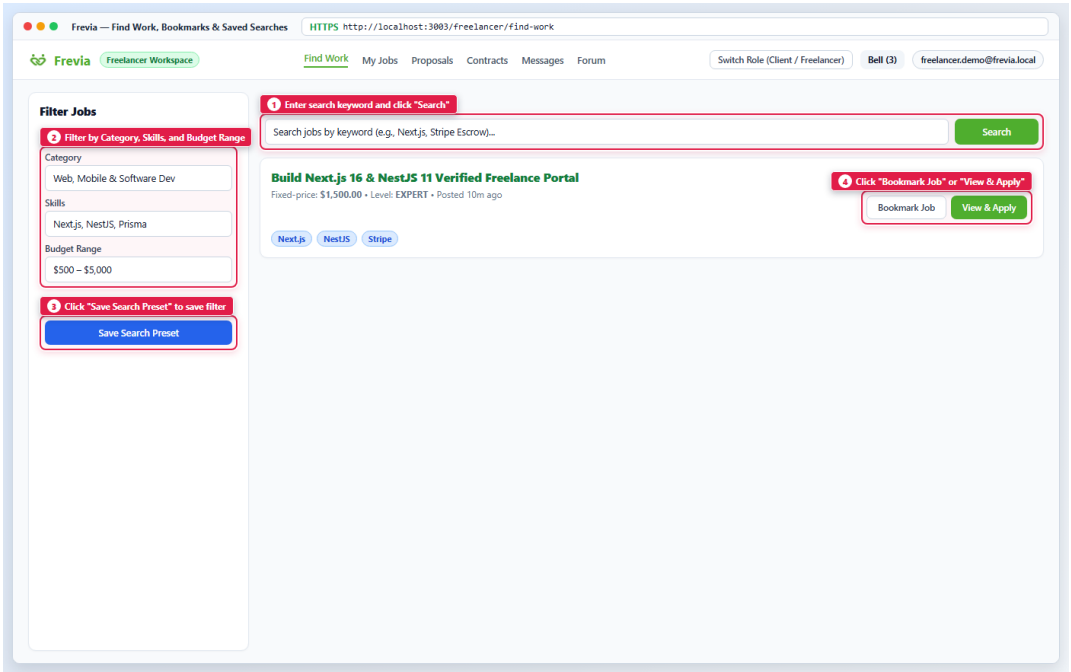


Figure 20. Freelancer job marketplace search, filters, and bookmarks

Notes and Common Errors:

- **Minimum Budget Validation:** Fixed-price and hourly budgets must be positive numeric values greater than zero, and the maximum budget must be greater than or equal to the minimum budget.
- **Closing a Job:** Clients can close an open job at any time from `/client/jobs/[jobId]`, preventing new proposal submissions.

3.6 Workflow 5: Proposal Submission & Candidate Hiring

Purpose and Roles: This workflow describes how a Freelancer submits a competitive proposal with proposed milestones for an open job and how a Client reviews candidates and accepts a proposal to automatically create a contract. **Involved Roles:** Freelancer, Client.

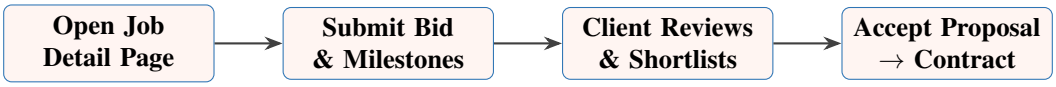


Figure 21. Proposal submission and candidate hiring workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Open Job Details (Freelancer):** From `/freelancer/find-work`, click a job title to open `/freelancer/jobs/[slug]` and click **Apply Now / Submit Proposal**.
2. **Step 2 – Fill Proposal Details:** Enter your **Bid Amount**, select **Estimated Duration**, write a tailored **Cover Letter**, answer any client **Screening Questions**, define the proposed **Milestones** (title, amount, due date), and upload supporting attachments.
3. **Step 3 – Submit Proposal:** Click **Submit Proposal**. You can track, edit, or withdraw pending proposals at `/freelancer/proposals`.
4. **Step 4 – Review Proposals (Client):** In the **Client** workspace, navigate to `/client/jobs/[jobId]/prop` Inspect each candidate's bid, cover letter, verified status, and portfolio.
5. **Step 5 – Shortlist, Reject, or Accept:** Click **Shortlist** to mark top candidates, **Message** to start an interview chat, or **Accept & Hire** on the winning proposal. Accepting a proposal automatically transitions it to `ACCEPTED` and generates the corresponding contract and milestones.

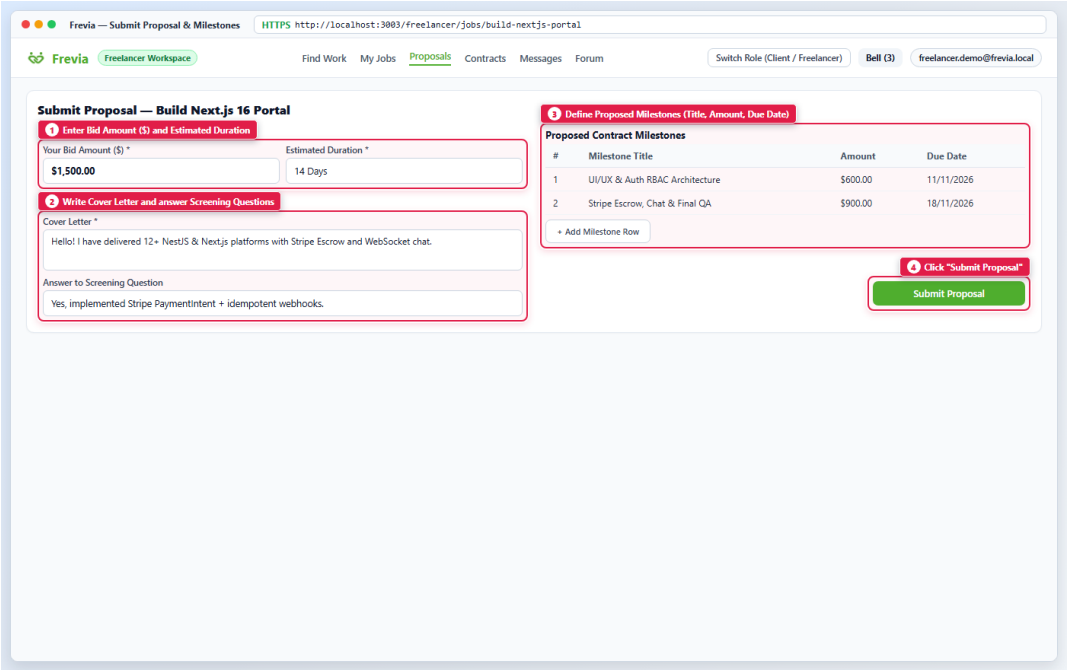


Figure 22. Freelancer proposal submission form with milestone breakdown

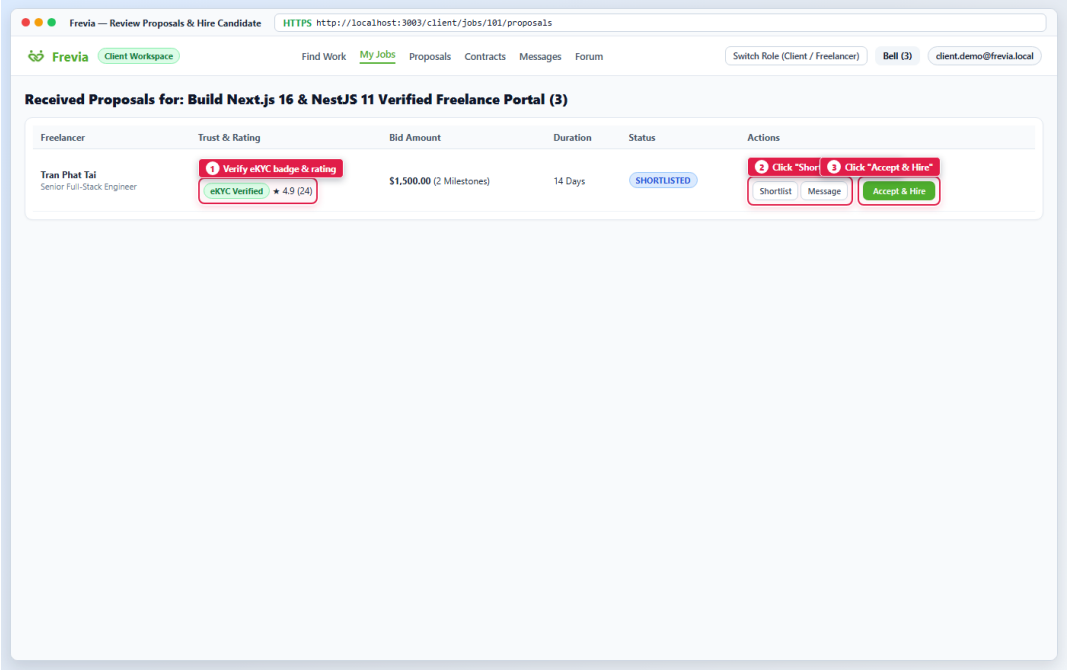


Figure 23. Client proposal comparison and candidate hiring screen

Notes and Common Errors:

- **One Active Proposal per Job:** A freelancer can submit only one active proposal per job post.
- **Self-Bidding Restriction:** Users cannot submit a proposal to a job post that they created themselves under their Client role.

3.7 Workflow 6: Contract Creation & Escrow Payment

Purpose and Roles: This workflow guides Clients through reviewing the generated contract, funding project milestones into Stripe Escrow, and activating the milestone for the Freelancer to begin work. **Involved Roles:** Client, Freelancer.

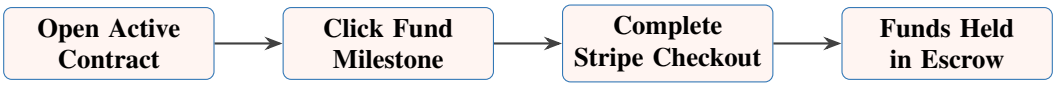


Figure 24. Contract creation and Stripe escrow payment workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Open Contract Workspace (Client):** Navigate to `/client/contracts` and click on the newly created contract to open `/client/contracts/[contractId]`.
2. **Step 2 – Initiate Milestone Funding:** Locate the target milestone in the **Milestones** list and click **Fund Milestone (Escrow)**.
3. **Step 3 – Complete Stripe Payment:** In the Stripe payment modal, enter valid payment card details (e.g., Stripe test card `4242 4242 4242 4242`, future expiry date, any 3-digit CVC) and click **Pay & Deposit to Escrow**.
4. **Step 4 – Verify Escrow Status:** Upon Stripe webhook confirmation (`payment_intent.succeeded`), the payment status updates to `ESCAPED / HELD_IN_ESCROW` and the milestone becomes active for deliverable submission. Clients can view all transaction records at `/client/payments`.

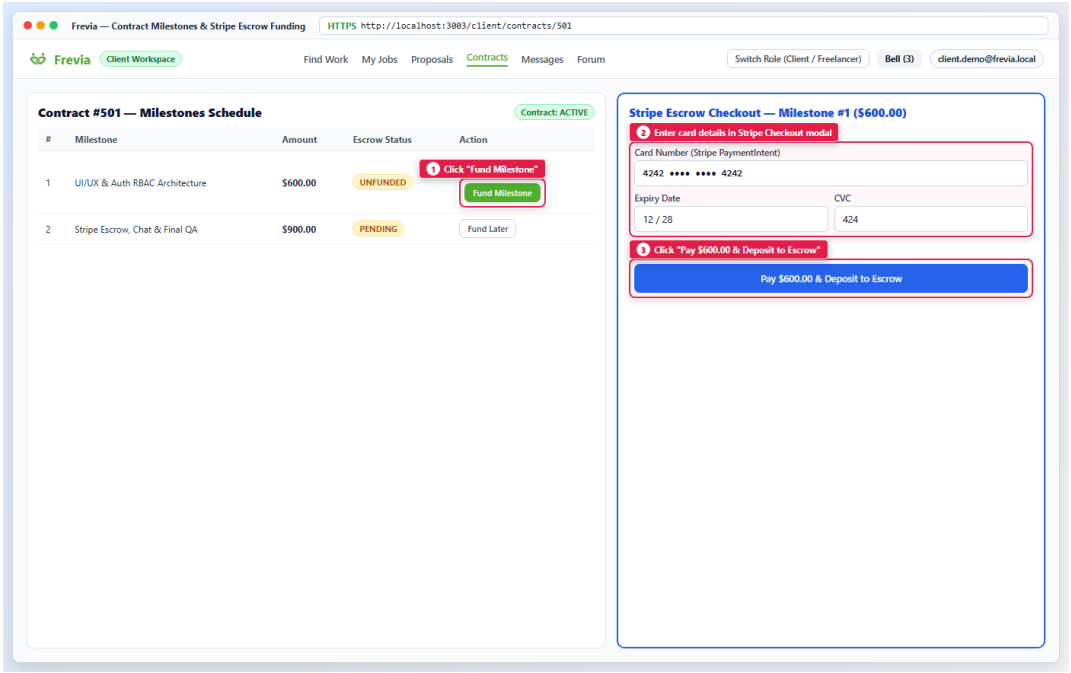


Figure 25. Contract milestone list and Stripe escrow funding checkout

Notes and Common Errors:

- **Unfunded Milestone Protection:** Freelancers are advised to wait until a milestone displays the **Funded in Escrow** status before delivering final work.

- **Payment Failure Handling:** If a card is declined, the milestone remains unfunded and the Client can retry payment immediately without duplicating the contract.

3.8 Workflow 7: Milestone Deliverable Submission & Fund Release

Purpose and Roles: This workflow details how Freelancers submit completed milestone deliverables and how Clients inspect the work, request revisions if needed, or approve the milestone to release escrow funds. **Involved Roles:** Freelancer, Client.

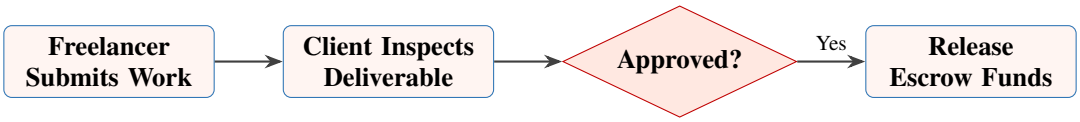


Figure 26. Milestone deliverable submission and escrow fund release workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Open Contract (Freelancer):** Navigate to `/freelancer/contracts/[contractId]` and select the funded milestone.
2. **Step 2 – Submit Deliverable:** Click **Submit Deliverable**, enter a detailed summary of the completed work, attach deliverable files or repository links, and click **Confirm Submission**. The milestone status changes to `SUBMITTED`.
3. **Step 3 – Inspect Deliverable (Client):** The Client receives a real-time notification and opens `/client/contracts/[contractId]` to download and review the submitted deliverables.
4. **Step 4 – Request Revision or Approve:**
   - If adjustments are required, click **Request Revision**, provide specific feedback notes, and submit. The milestone returns to `IN_PROGRESS`.
   - If the deliverable satisfies all requirements, click **Approve & Release Funds**.
5. **Step 5 – Escrow Fund Release:** Upon approval, the milestone transitions to `APPROVED / COMPLETED` and the escrowed funds are released to the Freelancer.

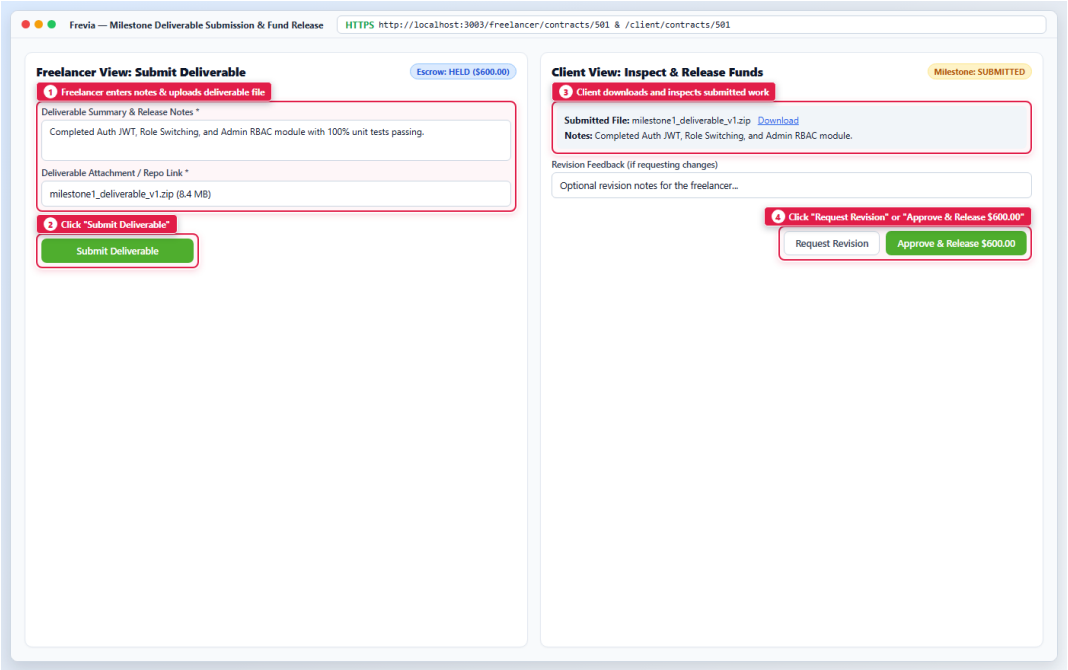


Figure 27. Freelancer deliverable upload and Client milestone approval screen

Notes and Common Errors:

- **Irreversible Fund Release:** Approving a milestone permanently releases the escrowed payment for that milestone and cannot be undone.

3.9 Workflow 8: Contract Dispute Resolution

Purpose and Roles: This workflow explains how either party opens a formal dispute when a contract disagreement occurs and how an Administrator investigates evidence and issues a binding financial resolution. **Involved Roles:** Client, Freelancer, Administrator.

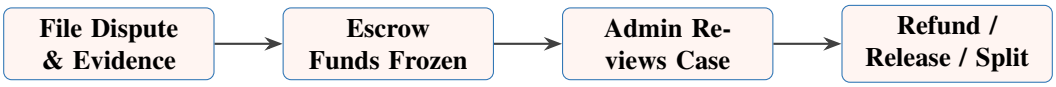


Figure 28. Contract dispute filing and administrative resolution workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Open Dispute Modal:** From the contract detail page (`/client/contracts/[contractId]` or `/freelancer/contracts/[contractId]`), click **Open Dispute** on the contested milestone.
2. **Step 2 – Submit Dispute Details:** Select the **Dispute Reason**, enter a comprehensive description of the issue, attach supporting evidence, and click **Submit Dispute**. Both parties can track the case at `/client/disputes` or `/freelancer/disputes`.
3. **Step 3 – Admin Investigation:** Sign in as an Administrator and navigate to `/admin/disputes`. Select the open dispute case to review the contract terms, milestone submissions, uploaded evidence, and conversation history.

4. **Step 4 – Execute Resolution:** Click **Resolve Dispute**, select the decision outcome (**Refund Client**, **Release to Freelancer**, or **Split Settlement**), enter the official resolution note, and click **Confirm Resolution**.

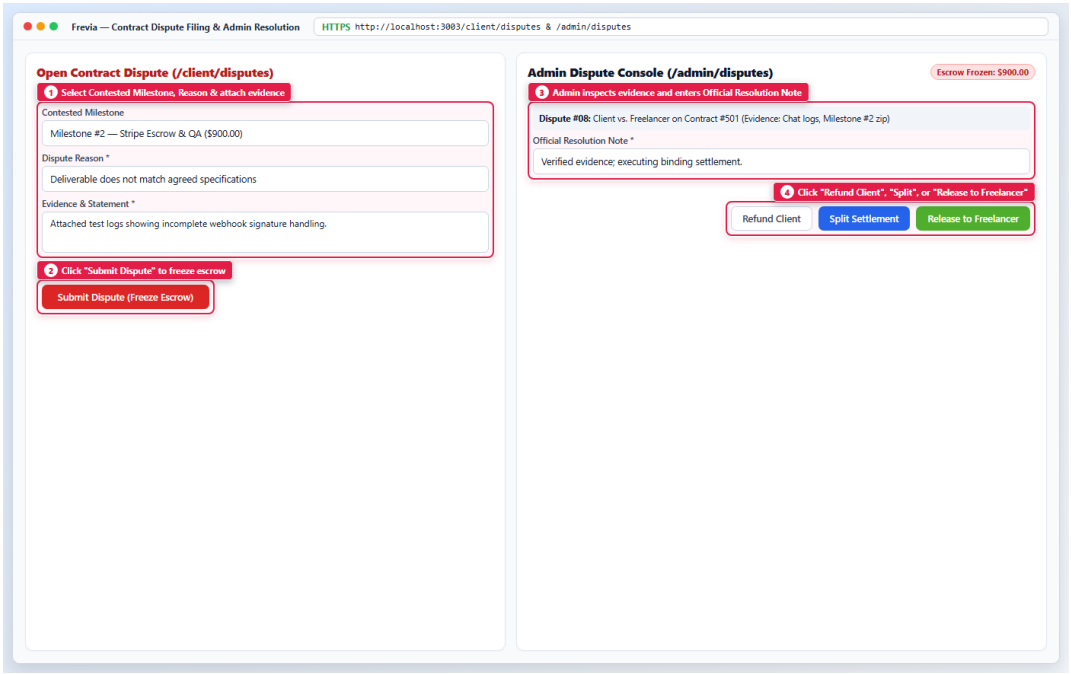


Figure 29. Contract dispute submission form and Admin dispute resolution console

Notes and Common Errors:

- **Escrow Freeze:** While a dispute is in `OPEN` or `UNDER_REVIEW` status, milestone funds are locked and cannot be released or refunded until the Administrator resolves the case or the initiator cancels the dispute.

3.10 Workflow 9: Ratings & Feedback

Purpose and Roles: This workflow guides Clients and Freelancers through submitting mutual star ratings and written reviews upon completing a contract. **Involved Roles:** Client, Freelancer.

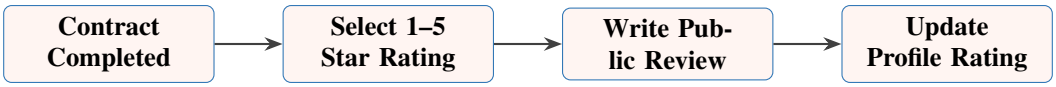


Figure 30. Two-way contract rating and feedback workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Open Completed Contract:** Navigate to `/client/contracts/[contractId]` or `/freelancer/contracts/[contractId]` once all milestones are completed and the contract status is `COMPLETED`.
2. **Step 2 – Rate Counterparty:** Click **Leave Review**, select a star rating from **1 Star** to **5 Stars**, and enter your written feedback describing quality, communication, and adherence to schedule.

3. **Step 3 – Submit Review:** Click **Submit Review**. The review is recorded and the counterparty's average rating and total review count are recalculated on their public profile (`/profiles/[id]`).

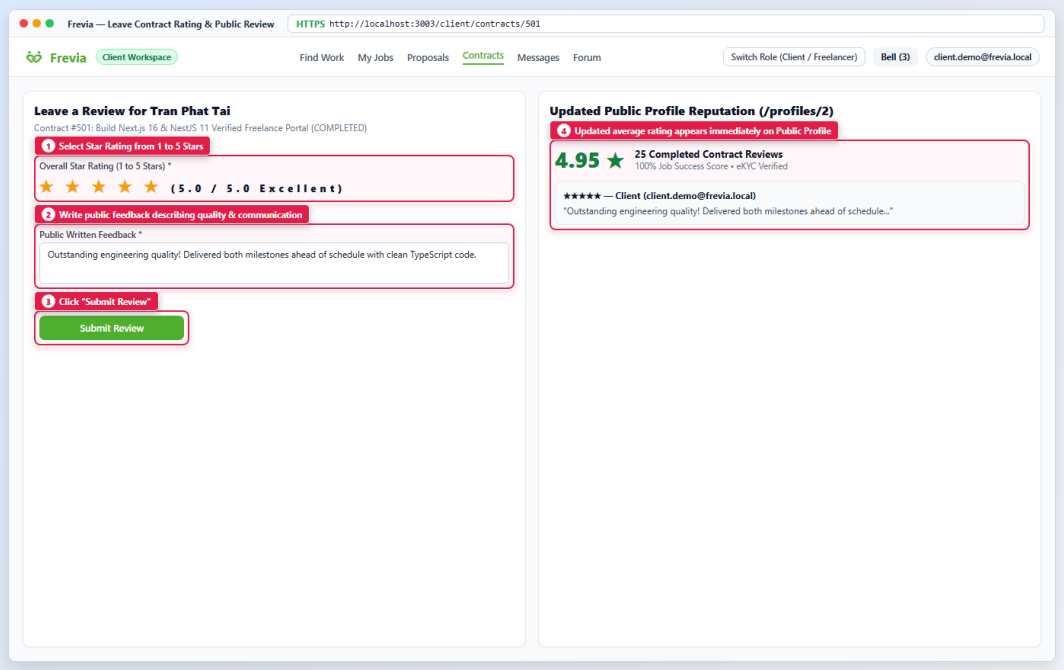


Figure 31. Two-way star rating and contract feedback submission modal

Notes and Common Errors:

- **Single Review Constraint:** Each party can submit at most one review per completed contract.

3.11 Workflow 10: Real-time Messaging & Notifications

Purpose and Roles: This workflow demonstrates how users communicate in real time via WebSocket chat rooms, share files, and manage live platform notifications. **Involved Roles:** Client, Freelancer, Moderator, Administrator.

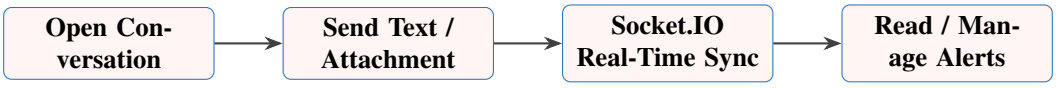


Figure 32. Real-time WebSocket messaging and notification workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Open Conversations Workspace:** Navigate to `/conversations` (or `/client/conversations` / `/freelancer/conversations`) and select a conversation thread from the left sidebar.
2. **Step 2 – Send Messages and Attachments:** Type your message in the composer, optionally click the **Attachment** icon to upload images or documents, and press **Enter** or click **Send**.
3. **Step 3 – Reply, Edit, or Delete Message:** Hover over any message bubble to click **Reply** (threaded quote), or click **Edit / Delete** on your own sent messages.

4. **Step 4 – Manage Real-Time Notifications:** Click the **Bell** icon in the top header or navigate to `/notifications`. Click any notification to jump directly to the associated proposal, contract, or message, or click **Mark All as Read**.

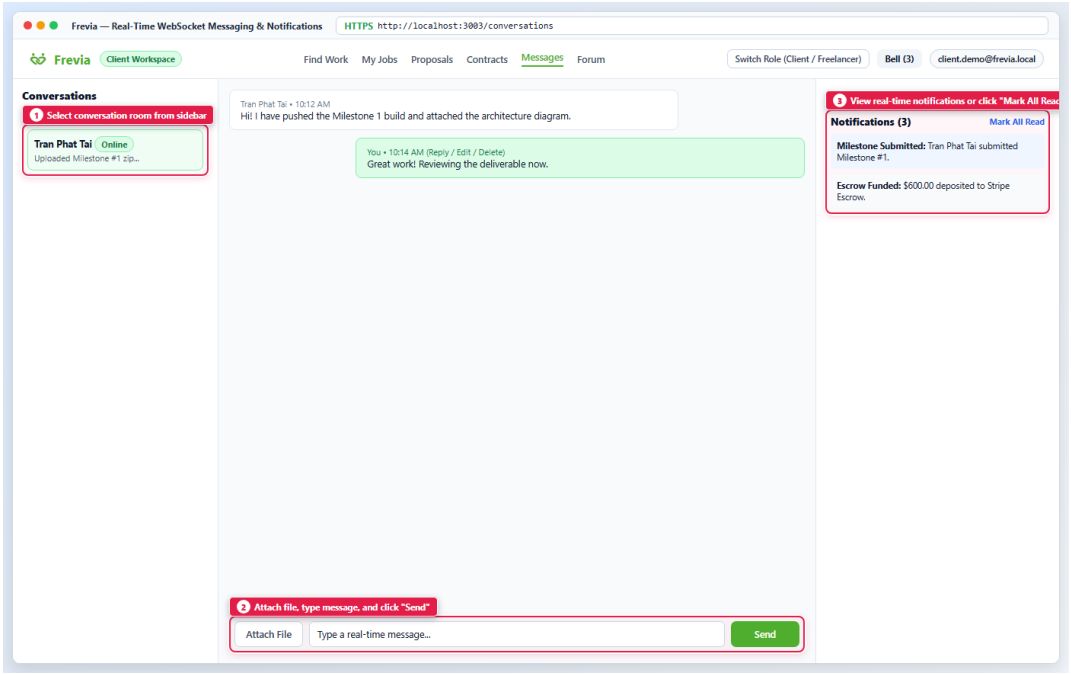


Figure 33. Real-time WebSocket conversation workspace and notification panel

Notes and Common Errors:

- **Participant Authorization:** Only verified participants of a conversation thread can view or send messages in that room.

3.12 Workflow 11: Community Forum & Content Moderation

Purpose and Roles: This workflow covers how community members publish discussion posts and comments in the forum, report inappropriate content, and how Moderators/Administrators review reports and manage soft-deleted items in the Forum Trash. **Involved Roles:** Client, Freelancer, Moderator, Administrator.

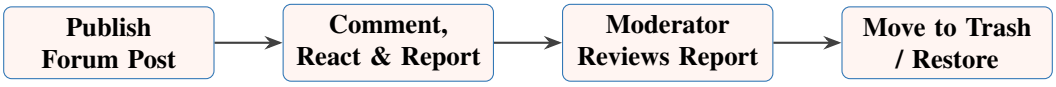


Figure 34. Community forum posting and content moderation workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Create Forum Post:** Navigate to `/forum`, click **Create Post**, select a **Forum Category**, enter the **Post Title**, write the content, add optional tags, and click **Publish Post**.
2. **Step 2 – Interact and Report:** Open any thread at `/forum/[categorySlug]/[postSlug]` to upvote/react, bookmark, or post a nested comment. If a post or comment violates community guidelines, click **Report**, select a reason, and submit.

3. **Step 3 – Moderate Posts and Reports (Moderator/Admin):** Navigate to `/admin/moderation` and `/admin/reports`. Inspect flagged posts or comments, approve pending posts, toggle **Pin**, or click **Delete (Move to Trash)** on violating content.
4. **Step 4 – Manage Forum Trash:** Navigate to `/admin/trash` to view soft-deleted posts and comments. Click **Restore** to recover an item back to the public forum, or click **Delete Permanently** to purge it from the database.

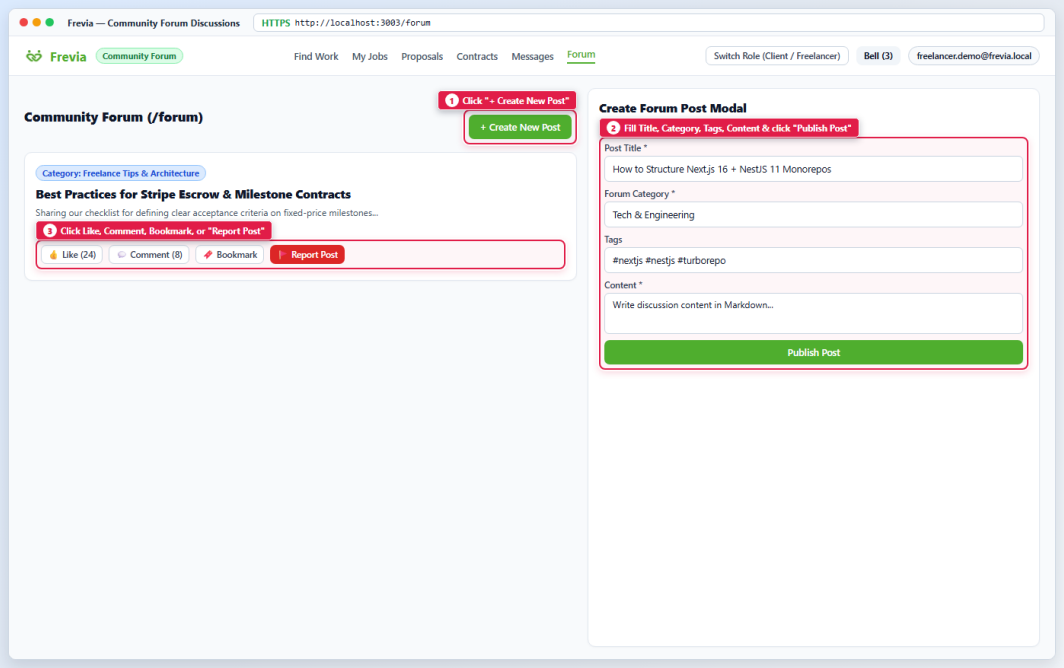


Figure 35. Community forum category view, discussion thread, and post editor

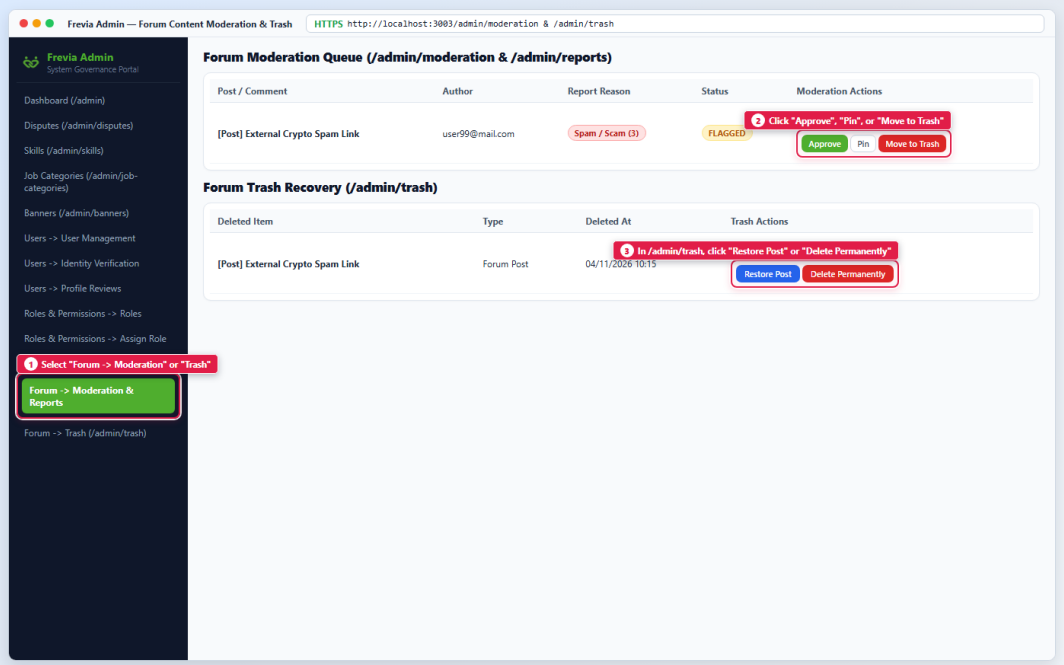


Figure 36. Admin/Moderator content moderation queue and forum trash management

Notes and Common Errors:

- **Soft-Delete Recovery:** Posts and comments deleted by authors or moderators are soft-deleted (`deletedAt` timestamped) and can be restored from `/admin/trash` until permanently purged.

3.13 Workflow 12: Admin System & Master Data Management

Purpose and Roles: This workflow guides Administrators through managing platform user accounts, dynamic RBAC roles and permissions, master skills, job categories, forum categories, and homepage advertisement banners. **Involved Roles:** Administrator.

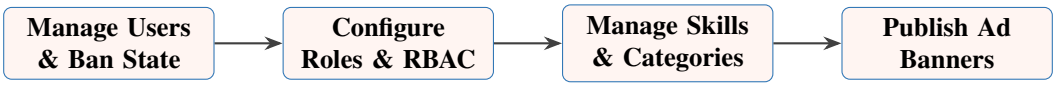


Figure 37. Administrative system and master data governance workflow diagram

Step-by-Step Instructions:

1. **Step 1 – Manage Users & Assign Roles (`/admin/users` & `/admin/assign-role`):** Search users by name or email, filter by role or ban status, click **Create User** to provision an account, click **Ban / Unban** to restrict or restore platform access, or assign/remove roles for any user.
2. **Step 2 – Configure RBAC Roles & Permissions (`/admin/roles` & `/admin/permissions`):** Create custom administrative roles, inspect HTTP endpoint permissions (`method`, `path`, `module`), and check/uncheck permissions assigned to each role.
3. **Step 3 – Manage Master Skills (`/admin/skills`):** View active and soft-deleted skills. Click **Create Skill** to add a new skill (**Name**, **Slug**, **Description**), click **Edit** to update details, click **Delete** to soft-delete, or click **Restore** to reactivate a deleted skill.
4. **Step 4 – Manage Job Categories (`/admin/job-categories`):** Create, update, soft-delete, or restore marketplace job categories (**Name**, **Slug**, **Description**) after verifying associated jobs.
5. **Step 5 – Manage Forum Categories (`/admin/categories`):** Create, edit, reorder, or delete community forum categories.
6. **Step 6 – Manage Advertisement Banners (`/admin/banners`):** Click **Create Banner**, enter the **Banner Title**, upload or specify the **Image URL**, set the target **Link URL**, configure **Start Date**, **End Date**, and **Active** toggle, and click **Save Banner**.

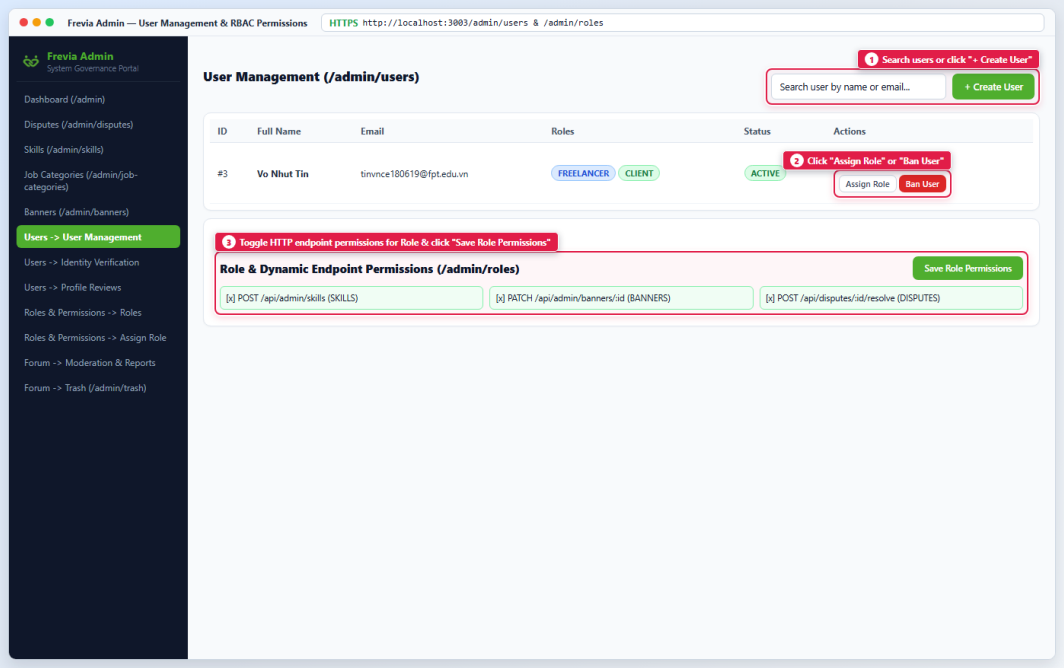


Figure 38. Admin user management directory and RBAC role-permission matrix

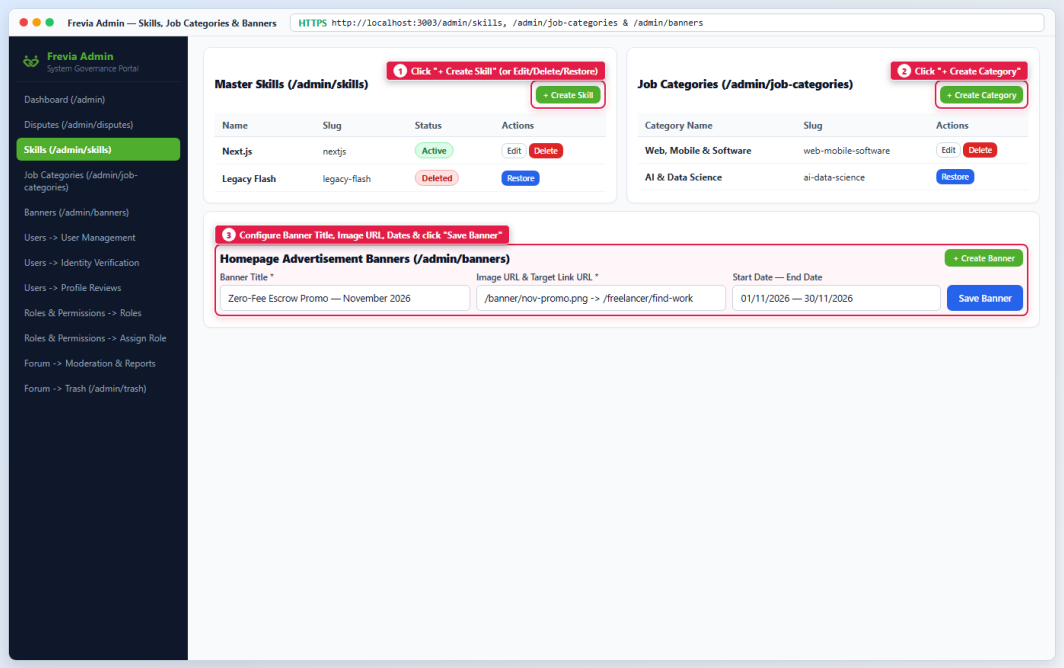


Figure 39. Admin management consoles for skills, job categories, and advertisement banners

Notes and Common Errors:

- **Unique Slug Enforcement:** Skill slugs, job category slugs, and forum category slugs must be unique in lowercase kebab-case format (e.g., `web-development`).
- **Self-Deletion Protection:** Administrators cannot delete or ban their own currently authenticated administrator account.